

Explicación Técnica Paso a Paso: ¿Cómo Funciona el Backend?

Este documento detalla técnicamente qué ocurre "detrás de escena" en el código de tu aplicación de Triage Preventivo, desde que se cargan los datos históricos hasta que el sistema emite una recomendación clínica en tiempo real.

Arquitectura General del Sistema

El proyecto está diseñado bajo una arquitectura modular y estructurada en capas bien definidas:

1. **Capa de Datos:** Contiene el dataset inicial (`synthetic_medical_triage.csv`) y los archivos serializados del modelo (`modelo_triage.pkl`).
 2. **Capa de Procesamiento y Machine Learning** (`reglas/modelo_triage.py`): Realiza la limpieza, ingeniería de variables, entrenamiento y evaluación del modelo utilizando **Pandas** y **Scikit-learn**.
 3. **Capa de Exposición / API** (`main.py`): Expone las interfaces de usuario (HTML/CSS) y los endpoints JSON utilizando el framework **FastAPI** y validación de tipos con **Pydantic**.
-

Flujo de Funcionamiento Paso a Paso

A continuación se detalla la secuencia lógica completa que ejecuta la aplicación:

Paso 1: Carga, Limpieza e Imputación de Datos (Pandas)

Cuando el backend se inicia por primera vez, o cuando invocas la ruta de reentrenamiento, se lee el archivo de datos tabulares `DATA/synthetic_medical_triage.csv` empleando Pandas:

1. **Eliminación de duplicados:** Se descartan filas idénticas con `drop_duplicates()` para evitar sesgos en el entrenamiento.
 2. **Manejo de valores nulos e infinitos:**
 - Se reemplazan infinitos con valores nulos (`NaN`).
 - Los valores nulos en signos vitales y datos clínicos se completan con la **mediana** de su respectiva columna (`fillna(df[col].median())`). Esto es una buena práctica clínica ya que evita distorsiones que causaría la media debido a valores atípicos severos.
 - Los niveles de triaje nulos se completan con la **moda** (el valor de clase más frecuente).
 3. **Control de rangos extremos (Clipping):** Se limitan los valores a rangos físicamente posibles en humanos (ej. acotar la temperatura entre 30°C y 45°C mediante `.clip()`), previniendo errores de medición.
-

Paso 2: Ingeniería de Variables (Feature Engineering)

Para ayudar a que el Árbol de Decisión detecte mejores patrones, se crean **columnas derivadas** a partir de los datos originales utilizando operaciones vectorizadas de Pandas:

- **Indicadores de Riesgo Binarios:** Se crean columnas con valor `0` (normal) o `1` (riesgo):

- **riesgo_oxigenacion**: 1 si la saturación de oxígeno es menor a 92%, de lo contrario 0.
 - **riesgo_temperatura**: 1 si la temperatura corporal es inferior a 36°C o superior a 38°C (hipotermia o fiebre).
 - **riesgo_dolor**: 1 si el dolor es severo (mayor o igual a 7).
 - **riesgo_comorbilidad**: 1 si el paciente tiene 2 o más enfermedades crónicas concurrentes.
 - **Scores Clínicos Compuestos**:
 - **score_signos_vitales**: Sumatoria de banderas críticas de signos vitales (presión baja, taquicardia, desaturación y fiebre).
 - **score_riesgo_total**: Agrupa los scores clínicos con las comorbilidades y dolores reportados.
 - **Ratios de Relación**:
 - **ratio_fc_presion**: Relación entre la frecuencia cardíaca y la presión sistólica (un indicador hemodinámico clave para detectar estados de shock).
-

Paso 3: Normalización Z-Score

Para evitar que las variables con números más grandes (como la presión o la frecuencia cardíaca) tengan mayor peso injustificado que las variables pequeñas (como la temperatura o cantidad de enfermedades crónicas), se aplica una normalización **Z-Score**:

$$\text{Valor Normalizado} = \frac{\text{Valor Original} - \text{Media}}{\text{Desviación Estándar}}$$

El sistema calcula y guarda la **media** y la **desviación estándar** de cada variable clínica. Estos parámetros se almacenan para poder normalizar de forma idéntica a los pacientes que ingresen en tiempo real en el futuro, evitando el fenómeno de *Data Leakage* (fuga de datos).

Paso 4: Entrenamiento y Evaluación del Árbol de Decisión (Scikit-Learn)

Una vez procesado el dataset final:

1. **División Estratificada**: Los datos se dividen en un **80% para entrenamiento** y un **20% para testeo** (`train_test_split`). Se usa un criterio de estratificación para garantizar que la proporción de niveles de triaje (0, 1, 2 y 3) sea equivalente en ambos conjuntos.
 2. **Entrenamiento del Algoritmo**: Se instancia un clasificador de árbol de decisión `DecisionTreeClassifier(max_depth=5, random_state=42)` y se entrena (`.fit(X_train, y_train)`). La profundidad máxima de 5 evita el sobreajuste (*overfitting*), permitiendo al modelo generalizar correctamente frente a pacientes nuevos.
 3. **Evaluación de Precisión (Accuracy)**: Se predice sobre el 20% de test reservado y se calcula la precisión final (`accuracy_score`).
 4. **Persistencia del Modelo**: Se guarda todo en un archivo binario `DATA/modelo_triaje.pkl` usando `joblib.dump()`. El archivo incluye:
 - El objeto del modelo entrenado.
 - La precisión de validación lograda.
 - Las estadísticas de normalización (medias y desviaciones estándar).
 - La versión y las columnas de variables empleadas.
-

Paso 5: Ciclo de Evaluación de Pacientes en Tiempo Real

Cuando un médico completa el formulario en la web y presiona "Evaluar paciente":

1. **Validación de Datos (Pydantic):** La clase `PacienteInput` en `main.py` analiza que los campos sean válidos (ej. la edad debe estar entre 0 y 120, la saturación entre 50 y 100). Si hay errores, los rechaza inmediatamente y muestra una pantalla amigable de validación, garantizando la consistencia del sistema.
2. **Ingeniería de Variables e Normalización en Caliente:** El backend toma los datos del paciente único y calcula en tiempo real sus scores de riesgo, ratios e indicadores clínicos. Luego, normaliza sus valores basándose en las **medias y desviaciones del dataset de entrenamiento** guardadas en el `.pkl`.
3. **Predicción y Probabilidad:**
 - El modelo clasifica al paciente devolviendo un nivel de triaje base: `0` (bajo), `1` (medio), `2` (alto), `3` (crítico).
 - Se calcula la confianza de la predicción con `predict_proba()`. Por ejemplo, si el árbol estimó que el paciente es de prioridad Alta con un 93% de probabilidad, el sistema mostrará **93%** como puntaje de confianza.
4. **Regla de Ajuste Clínico (Override Experto):** Se aplica una regla heurística de seguridad: si el paciente llegó en **ambulancia** o **silla de ruedas**, el nivel se incrementa en uno (`nivel = min(nivel + 1, 3)`), priorizando la atención de pacientes que no pueden trasladarse por sus propios medios.
5. **Generación Dinámica de Motivos:** Compara cada constante vital del paciente con los umbrales fisiológicos saludables (ej. saturación menor a 95%, temperatura fuera de [36°C, 37.5°C]). Las anomalías detectadas son convertidas a enunciados textuales claros para respaldar el criterio del sistema.
6. **Guardado en Historial y Renderizado:** El registro resultante se añade a una lista global en memoria (`historial_triaje`) y se renderiza en la plantilla HTML usando Jinja2.